

Reviewer Pack — Quantum Phase of Cubic Forms Bounds Resolution Width

Entry 234a33e87f4f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 23:24:20 UTC

Quantum Phase of Cubic Forms Bounds Resolution Width

Entry ID: 234a33e87f4f **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-29 23:24:20 UTC

1. Conjecture

Field A (mathematical branch): Quantum Information Theory (Quantum Phases) **Field B** (complexity object): Boolean Function Complexity (Resolution Proof Width)

Statement:

For every boolean function f with resolution proof width $w(f)$, the quantum phase of its characteristic polynomial, defined as the absolute value of the leading coefficient modulo the order of the finite field used in the characteristic polynomial, satisfies $|\varphi_f| \leq c \log w(f)$ for some constant c .

Rationale (proposer's reasoning):

Quantum phases are known to be closely related to entanglement properties, and entanglement is a fundamental resource for quantum computation. A direct connection between quantum phases and resolution proof width could reveal new insights into the complexity of boolean functions and potentially lead to more efficient algorithms.

Taxonomy category: cg_kw_andreev (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 76aa5903436abe59

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a boolean function f , if $|\varphi_f| \leq c \log w(f)$ holds for all generated functions with a correlation coefficient close to 1 across 30 seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "quantum phase" cubic forms AND resolution proof width" - "characteristic polynomial" quantum information theory Boolean function complexity" - "leading coefficient modulo field order" AND "resolution width" boolean functions

Top relevant hits considered: - [http://arxiv.org/abs/0909.4822v1] Quantum Phase Transitions in Spin-Boson Systems: Dissipation and Light Phenomena - [http://arxiv.org/abs/1904.01603v1] Quantum phase properties of photon added and subtracted displaced Fock states - [http://arxiv.org/abs/0909.2316v2] Quantum Phase Transition, Dissipation, and Measurement - [http://arxiv.org/abs/2407.04826v1] Multi-strategy Based Quantum Cost Reduction of Quantum Boolean Circuits - [http://arxiv.org/abs/hep-th/9707234v2] Variational Approach to Quantum Field Theory: Gaussian Approximation and the Perturbative Expansion around It - [http://arxiv.org/abs/quant-ph/0305192v1] Photon engineering for quantum information processing

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_boolean_function(n, m):
    variables = list(range(1, n + 1))
    clauses = set()
    for _ in range(m):
        literals = [random.choice([1, -1]) * random.choice(variables) for _ in range(random.randint(1, n))]
        clauses.add(tuple(sorted(literals)))
    return variables, clauses

def characteristic_polynomial(clauses, variables):
    n = len(variables)
    poly = [[0] * (n + 1) for _ in range(n + 1)]
    poly[0][0] = 1
    for clause in clauses:
        product = 1
        for literal in clause:
            if literal > 0:
                product *= variables[literal - 1]
            else:
                product *= (1 - variables[-literal])
        for i in range(n, -1, -1):
            poly[i][0] += product
            for j in range(i):
                poly[j][i + 1] = poly[j + 1][i]
```

```

# Convert to integer coefficients modulo a large prime
prime = 2**64 - 59
for i in range(n + 1):
    for j in range(n + 1):
        poly[i][j] %= prime

return poly

def resolution_width(clauses, variables):
    n = len(variables)
    clauses_list = list(clauses)
    stack = []
    while clauses_list:
        clause = clauses_list.pop()
        if all(lit in variables for lit in clause):
            return len(stack) + 1
        literal = next((lit for lit in clause if -lit in variables), None)
        if literal is not None:
            stack.append(literal)
            new_clauses = set()
            for other_clause in clauses_list:
                if literal in other_clause and -literal in other_clause:
                    continue
                new_clauses.add(tuple(sorted(other_clause + (other_clause.index(-literal) + 1,))))
            clauses_list = new_clauses
        else:
            return float('inf')
    return len(stack)

def quantum_phase(poly):
    leading_coefficient = poly[0][0]
    order_of_field = 2**64 - 59
    return abs(leading_coefficient) % order_of_field

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    results = []
    for n in n_values:
        variables, clauses = generate_boolean_function(n, 2**n)
        poly = characteristic_polynomial(clauses, variables)
        w_f = resolution_width(clauses, variables)
        if w_f == float('inf'):
            continue
        phi_f = quantum_phase(poly)
        c_log_w_f = 10 * math.log(w_f) # Example constant c=10 for simplicity
        results.append({"n": n, "phi_f": phi_f, "c_log_w_f": c_log_w_f})

    if not results:
        return {
            "metric_name": "quantum_phase_bound",
            "metric_value": None,
            "instances_tested": 0,
            "n_max": 0,
            "conjecture_holds": False,

```

```

        "counterexample": "no_valid_instances"
    }

    phi_f_values = [result["phi_f"] for result in results]
    c_log_w_f_values = [result["c_log_w_f"] for result in results]

    mean_phi_f = sum(phi_f_values) / len(phi_f_values)
    std_phi_f = math.sqrt(sum((x - mean_phi_f) ** 2 for x in phi_f_values) / len(phi_f_values))

    correlation_coefficient = sum((phi_f_values[i] - mean_phi_f) * (c_log_w_f_values[i] - mean_c_l

    return {
        "metric_name": "quantum_phase_bound",
        "metric_value": correlation_coefficient,
        "instances_tested": len(phi_f_values),
        "n_max": max(result["n"] for result in results),
        "conjecture_holds": abs(correlation_coefficient) > 0.95, # Example threshold
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i - 1 for i in range(2, 32)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"first_failing_seed\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE no_valid_instances")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b5184e0d.py", line 123, in <module>

```

    result = run_trial(seed)
    ^^^^^^^^^^^^^^^^^

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b5184e0d.py", line 82, in run_trial

```

    poly = characteristic_polynomial(clauses, variables)

```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b5184e0d.py", line 40, in characteristic
    poly[j][i + 1] = poly[j + 1][i]
    ^^^^^^^
```

IndexError: list assignment index out of range

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete its execution to verify the conjecture. | next: Re-run the test with proper error handling and ensure that it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15212
2	preregistration	ollama_remote	glm4:latest	0	0	11304
3	novelty	ollama_remote	glm4:latest	0	0	19126
4	novelty	ollama_remote	glm4:latest	0	0	13945
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	23488
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	27727
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20524
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21780
9	judge	ollama_remote	glm4:latest	0	0	19365

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 172471 ms total latency. Provider mix: {'ollama remote': 9}

(full prompt+response transcripts available in [research/audit/234a33e87f4f.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/234a33e87f4f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp sandbox/test *.py` (per-cycle)

Replay tarball: `research/replay/234a33e87f4f.tar.gz` (if generated)